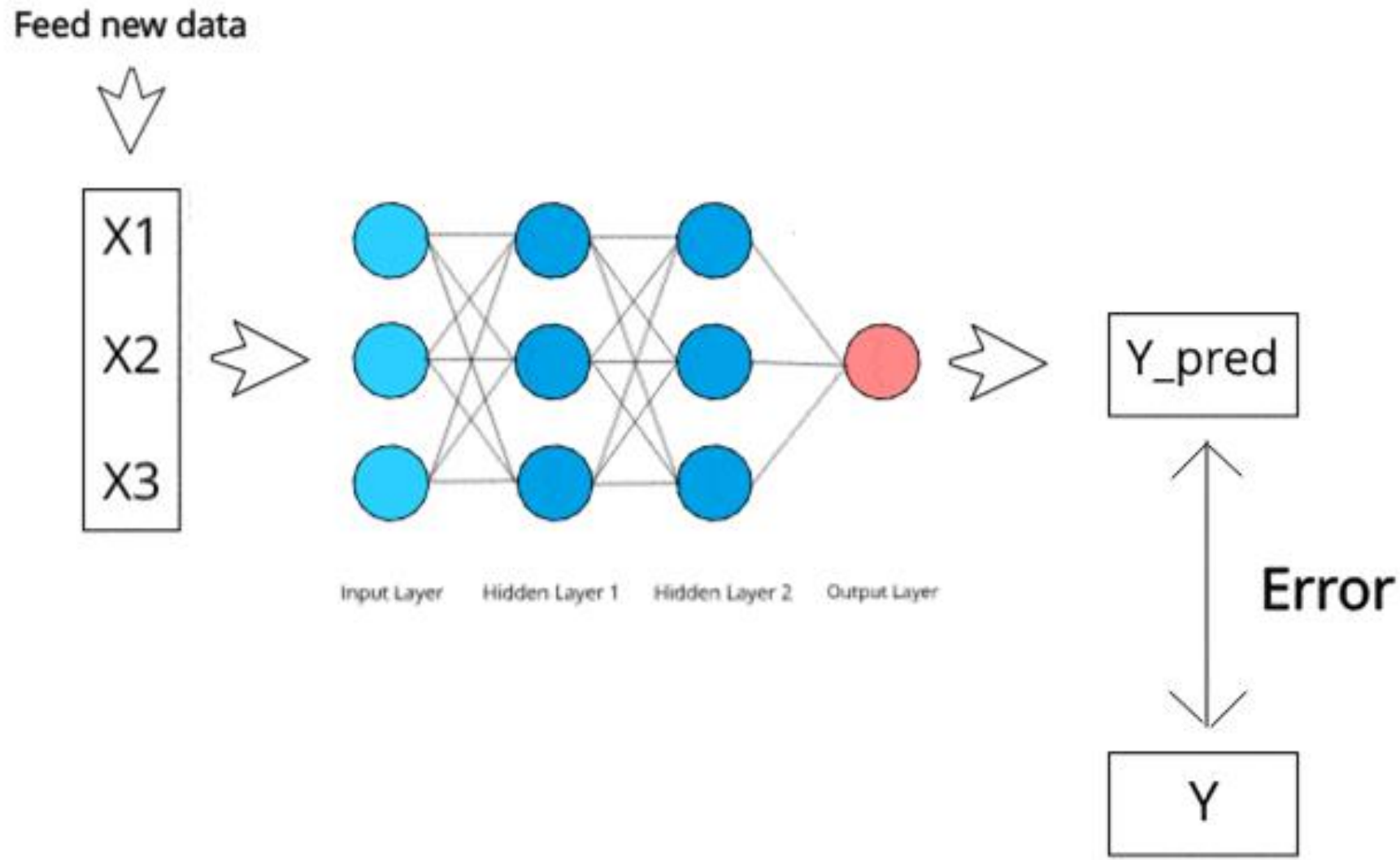
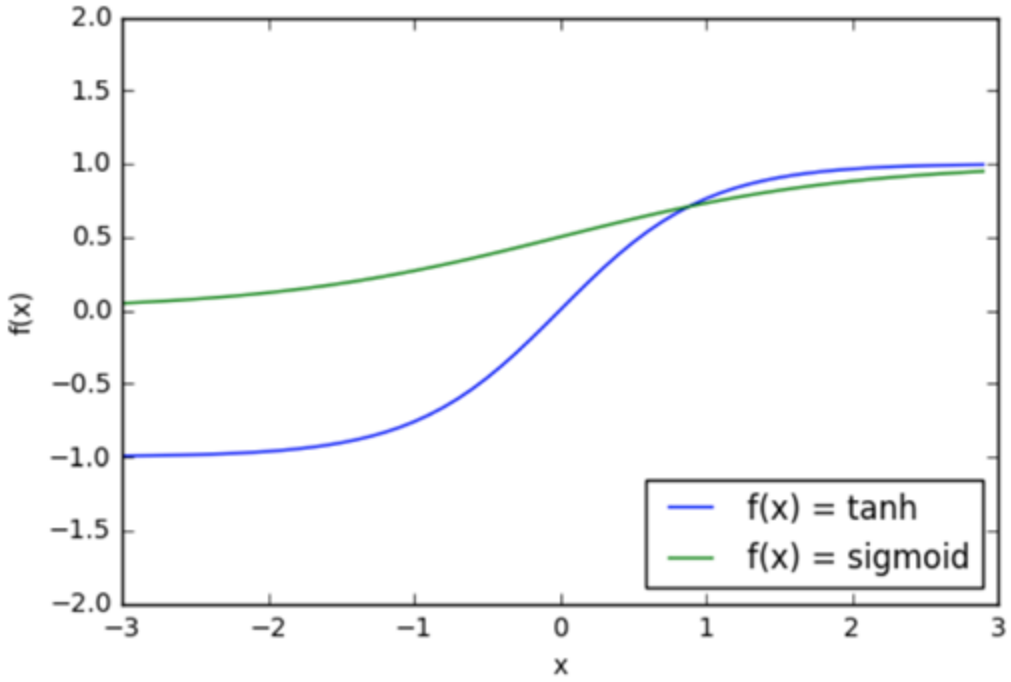
Deep Learning Course

Preventing Overfitting


Feed new data
X1
X2
X3
Input Layer
Hidden Layer 1
Hidden Layer 2
Output Layer
Y_pred
Error
Y

Hyperbolic Tangent Functions



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = a\ (1 - a)$$

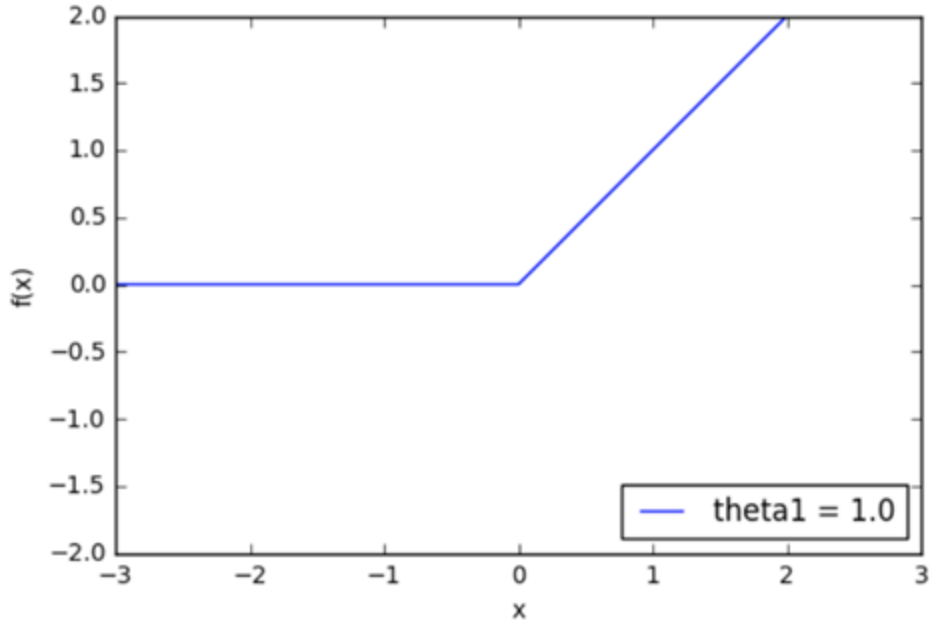
Logistic Activation Function + Derivative

$$g(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

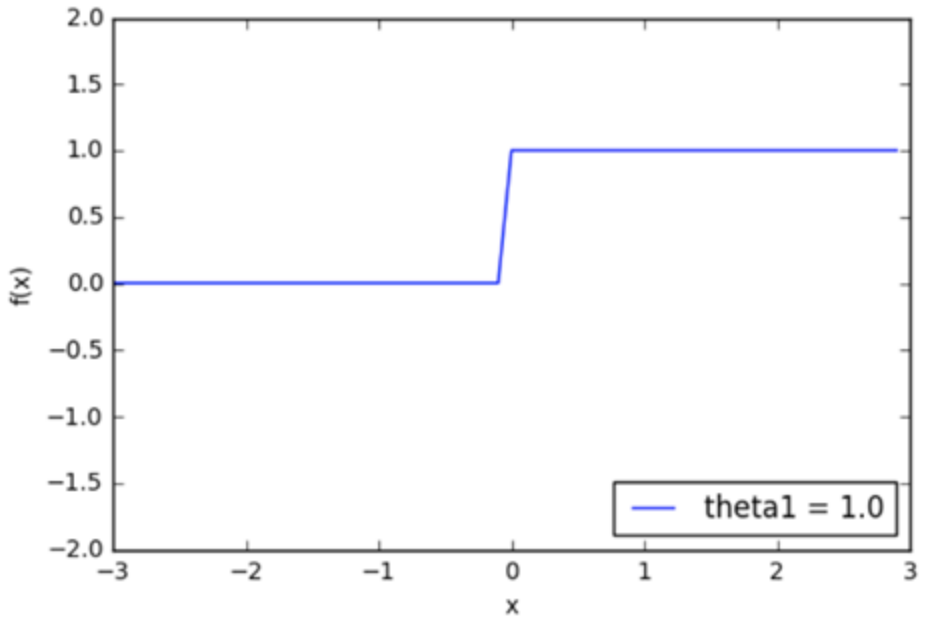
$$g'(z) = 1 - tanh^{2}(z)$$

HTAN Activation Function + Derivative

Rectified Linear Units

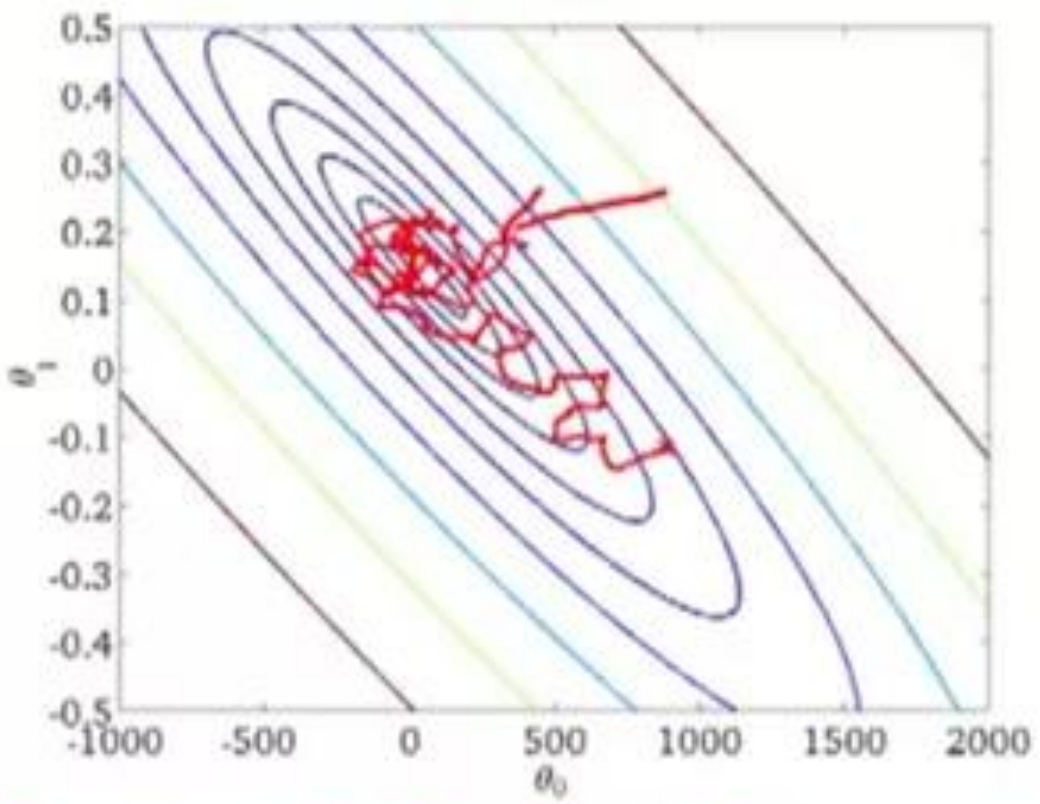


$$g(z) = max(0, z)$$



$$g'(z) = \begin{cases} 0 & if\ g(z) <= 0 \\ 1 & if\ g(z) > 0 \end{cases}$$

Stochastic Gradient Descent



Update after each individual example.

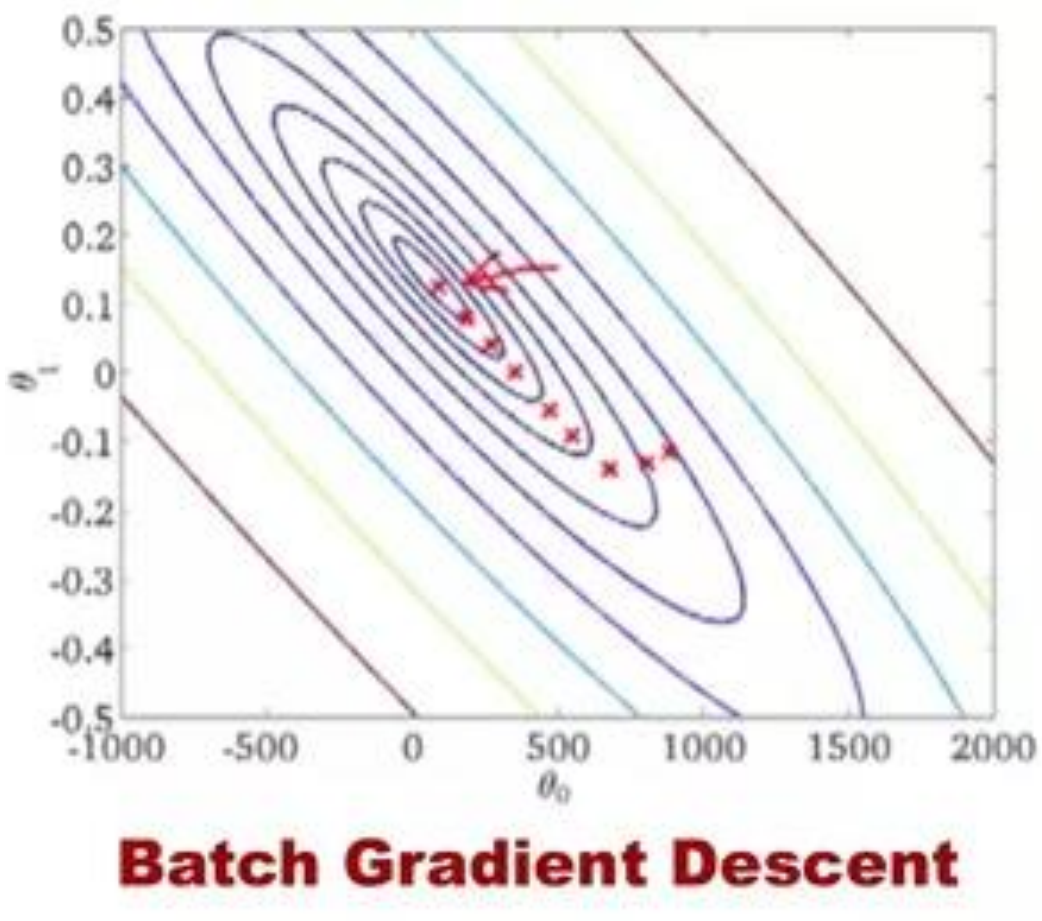
-- this is in contrast to full gradient descent where we look at all of our examples and calculate a total loss before applying gradients

Pros: Convergence is better
Cons: Quite Expensive

Stochastic Gradient Descent (with Batches)

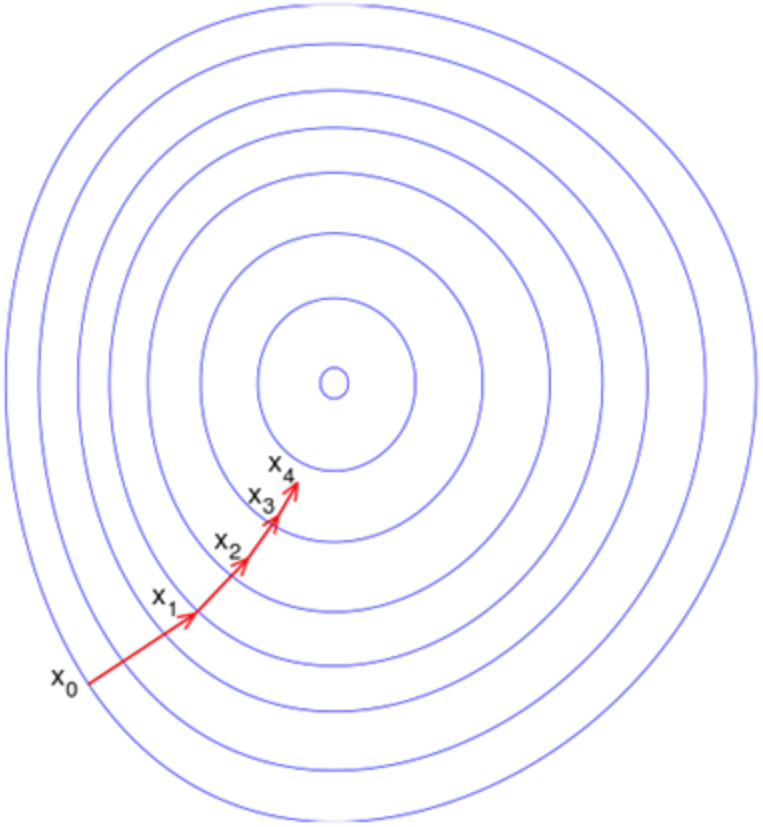
Happy middle ground.

Save up losses from a number of examples (a batch) before applying the updates.



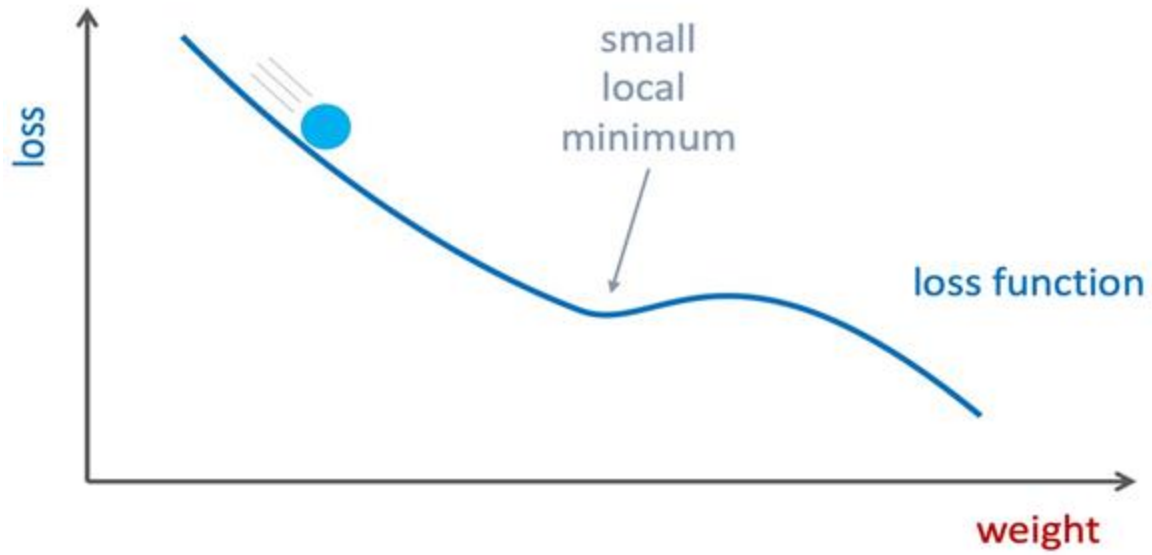
Improving Optimizers

Adjustable Learning Rates



Improving Optimizers

Momentum



momentum

Improving Optimizers

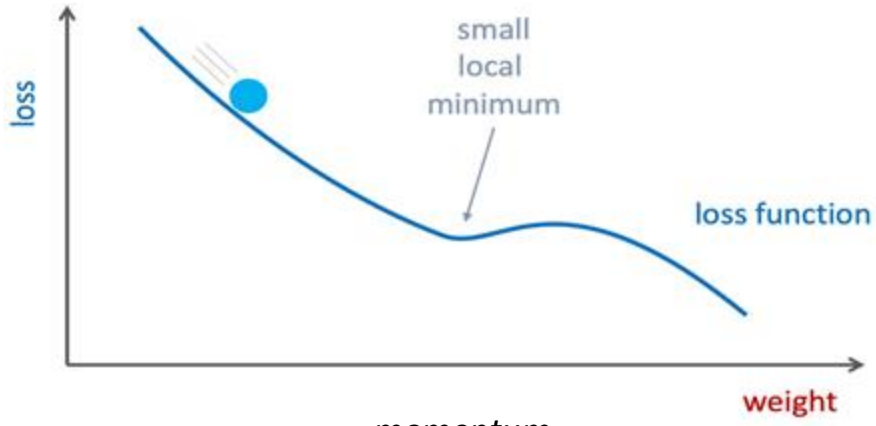
Feature Specific Learning Rates



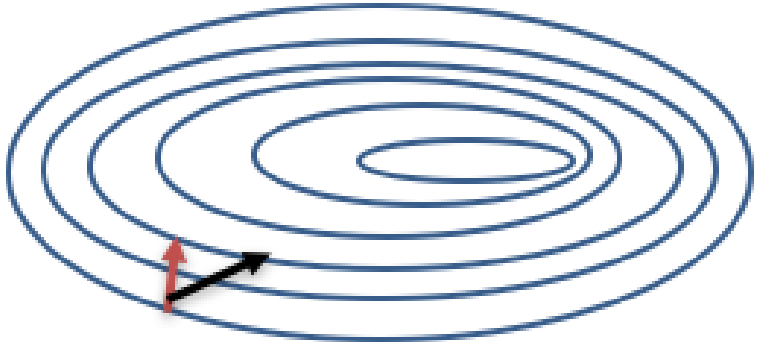
multiple error rates

Improving Optimizers

- Principles
  - Adjust Learning Rate
  - Use some notion of momentum for overcoming local minima
  - Keep a different error rate for each feature (dimension)



momentum



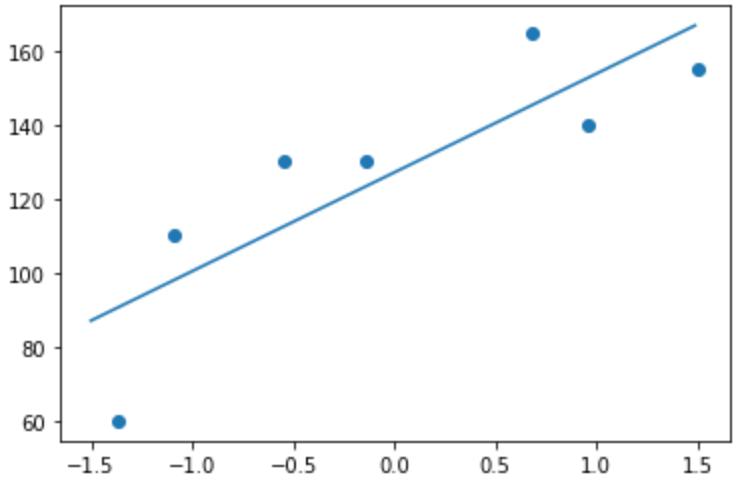
multiple error rates

State of Art Optimizers

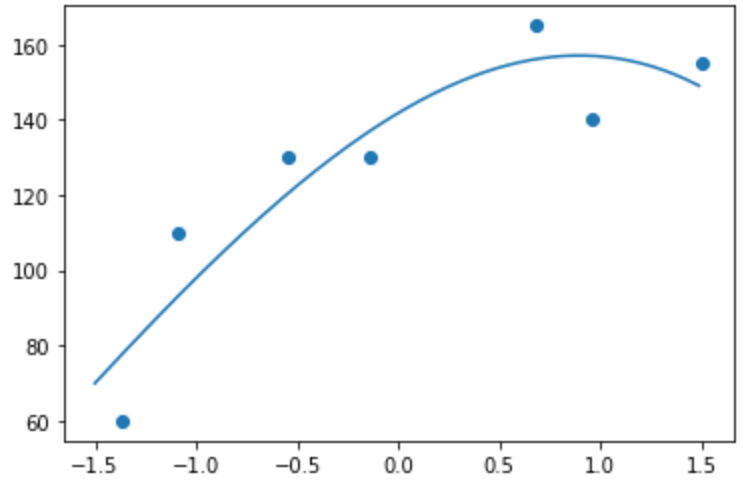
- Key Examples
  - Adam
  - Adagrad
  - RMSProp –These are all variations on a theme
    - Using momentum
    - Using multiple learning rates
    - Adjusting learning rates on the fly
- Analytical Approaches (optional)
  - LBFGS – Not seriously used in practical NN applications

This Week:
Preventing Overfitting

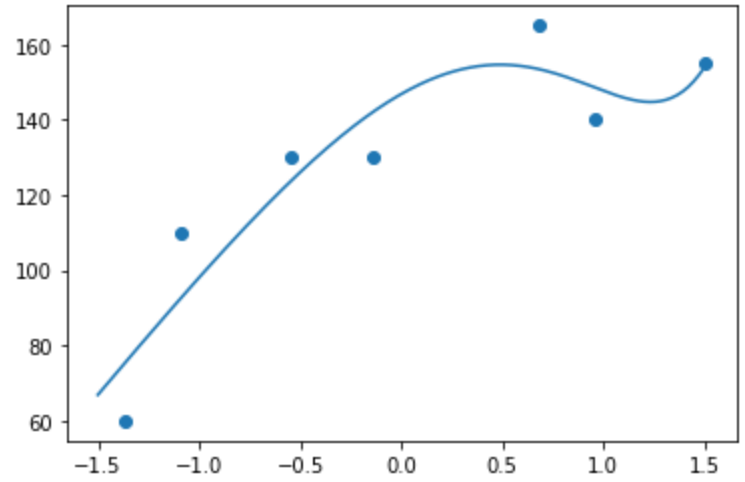
The Overfitting Problem - Example: Linear Regression



Few Parameters - Reasonable Fit

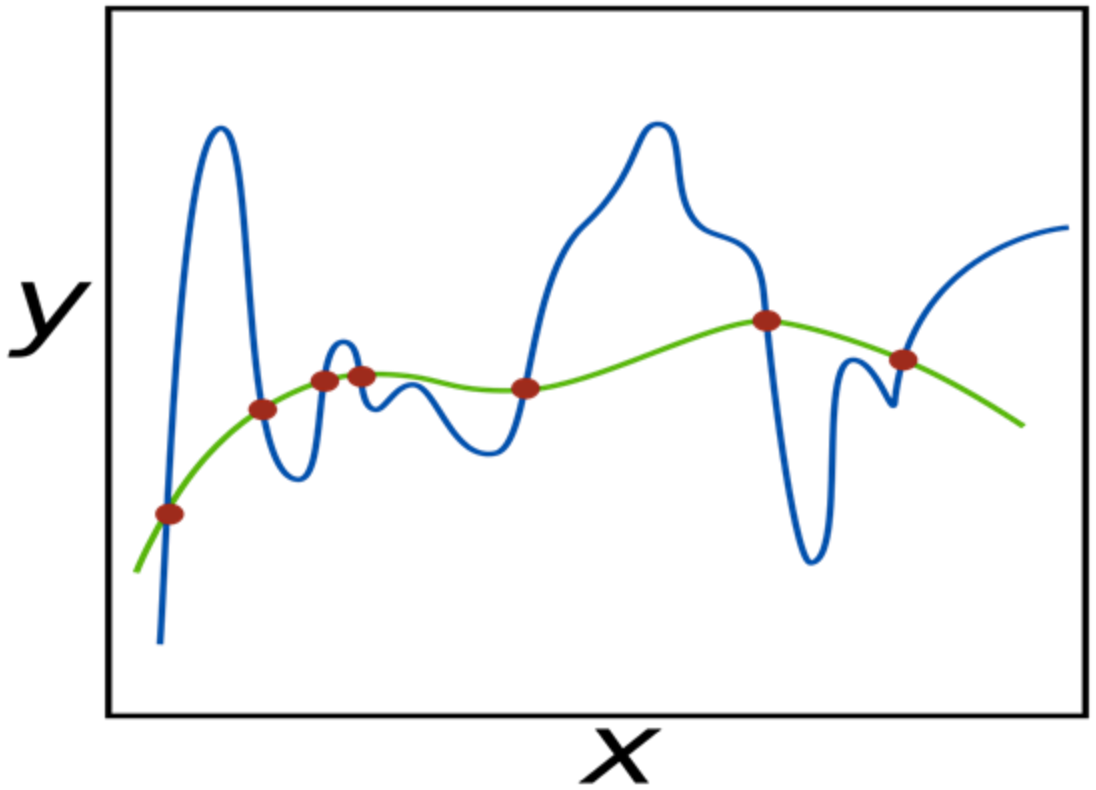


More Parameters - Better Fit

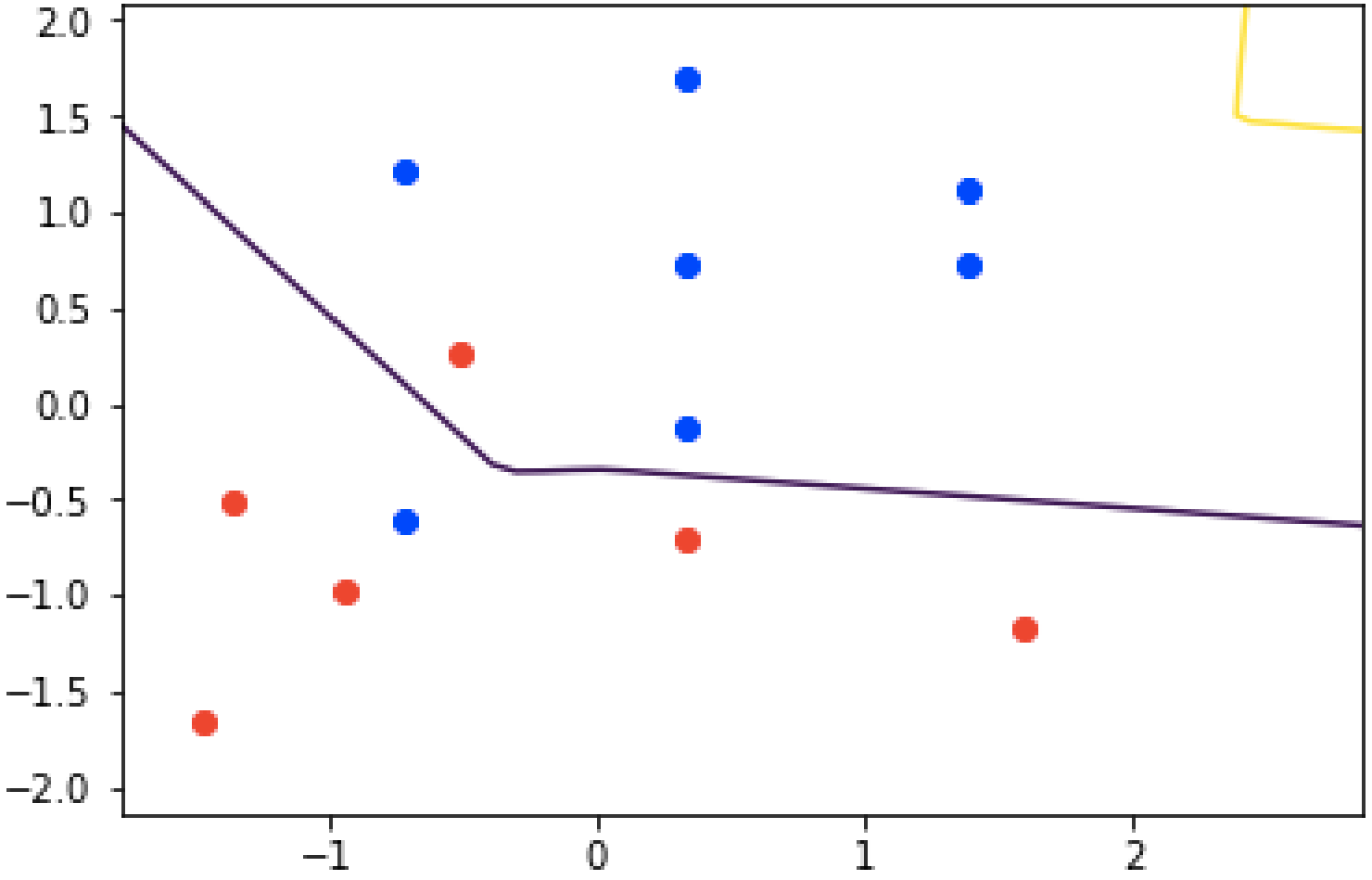


Even More Parameters - Going too far????

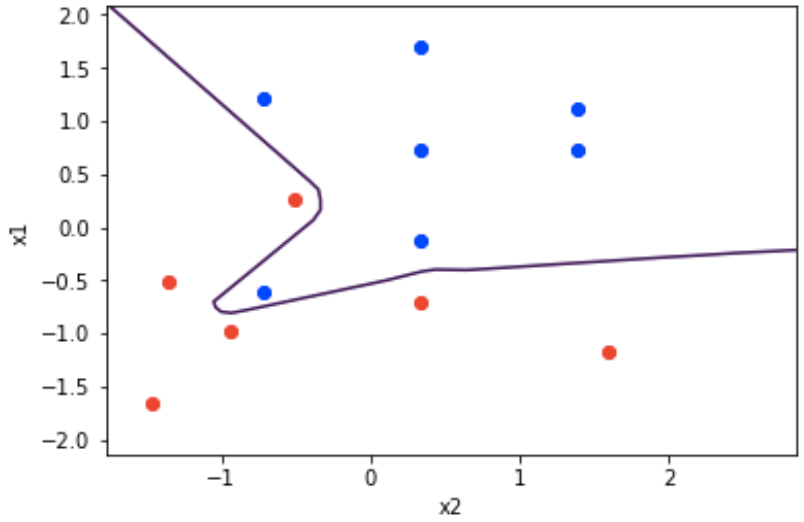
Overfitting Extreme Regression Case



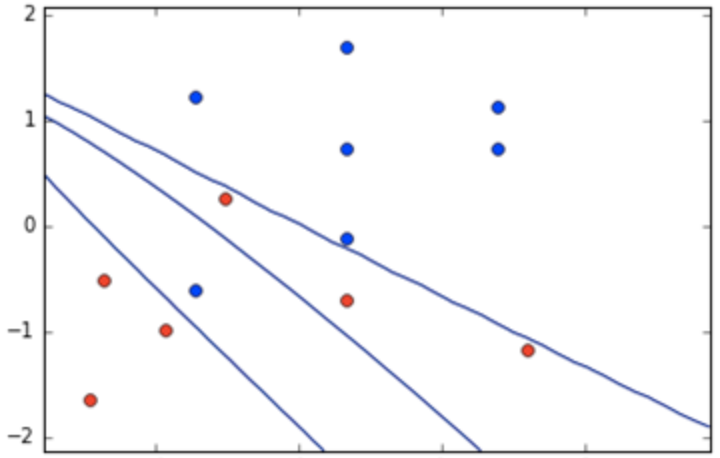
Overfitting in classification



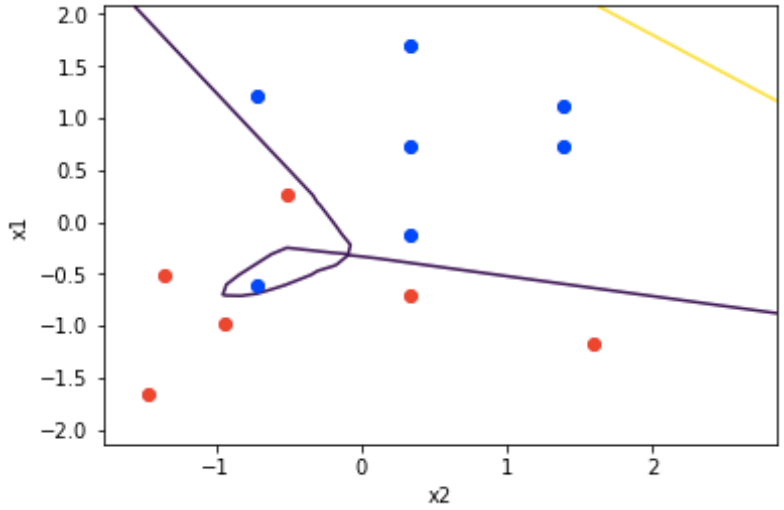
Overfitting on a NN Classification Task



Overfitting

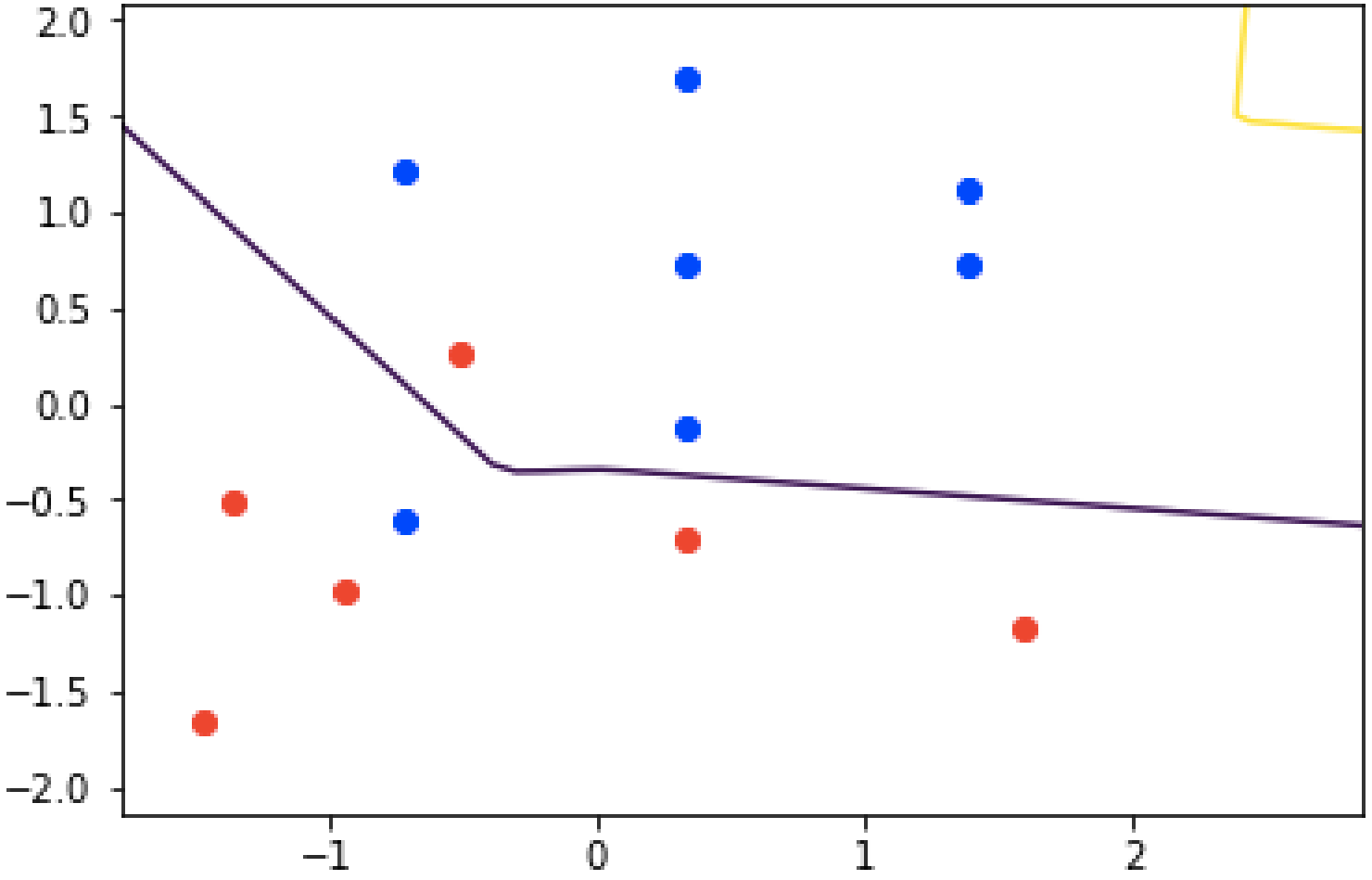


Horrible Overfitting



A ‘better’ solution

Overfitting - Finding the Sweet Spot



Method 1: Regularization

$$\frac{\lambda}{2}\sum_{k}\theta_k^2$$

Training_Cost = Actual_Error + Penalty_for_High_Weights

e.g., Sum of Squared Error or Cross Entropy Loss Function

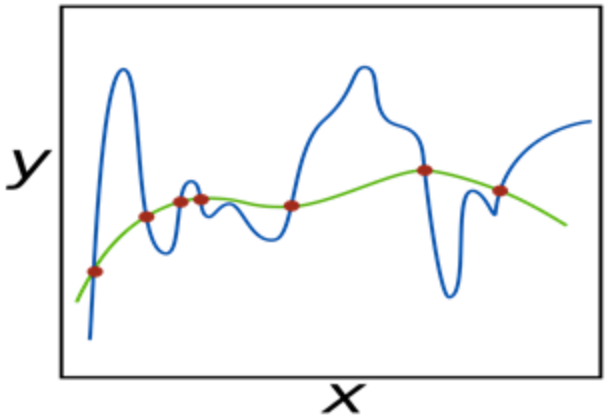
Regularisation

Instead of just making our cost proportional to our actual error on training data, we can use a cost function which includes a part which is proportional to the magnitude of parameters.

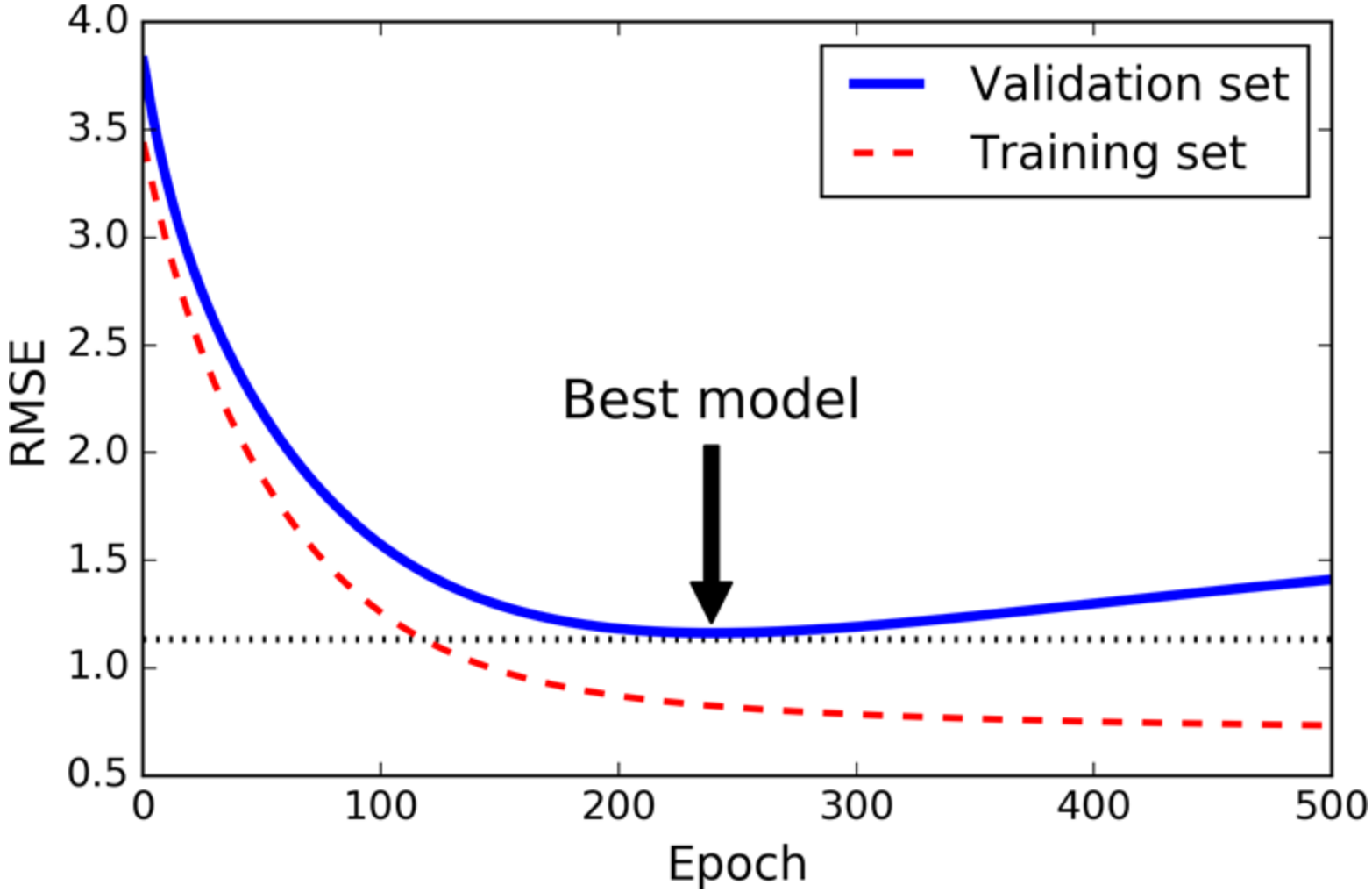
Regularisation penalizes extreme parameters and in so doing helps to stop overfitting.

$$C = \frac{1}{2}\sum_i (y_i - a_i)^2 + \frac{\lambda}{2}\sum_k \theta_k^2$$

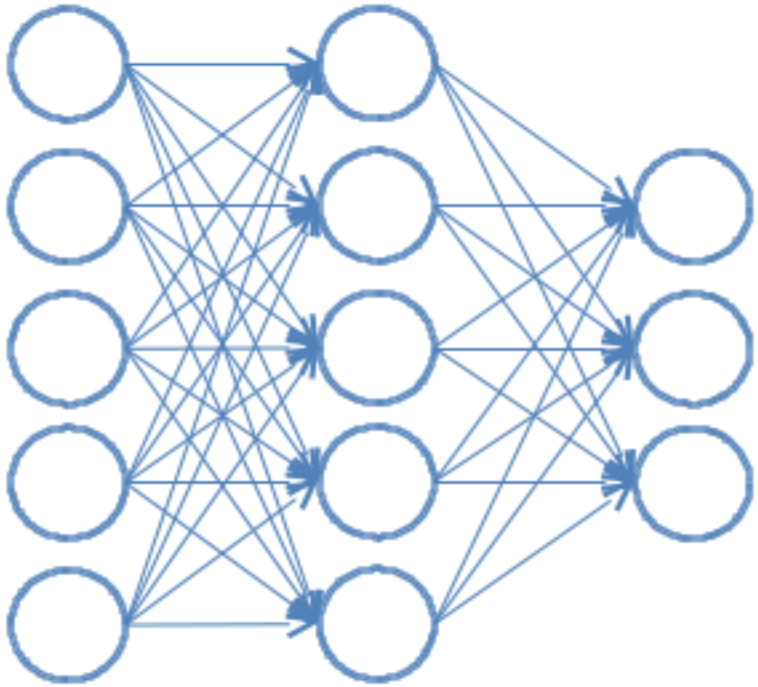
Cost function combining MSE and L2 regularisation



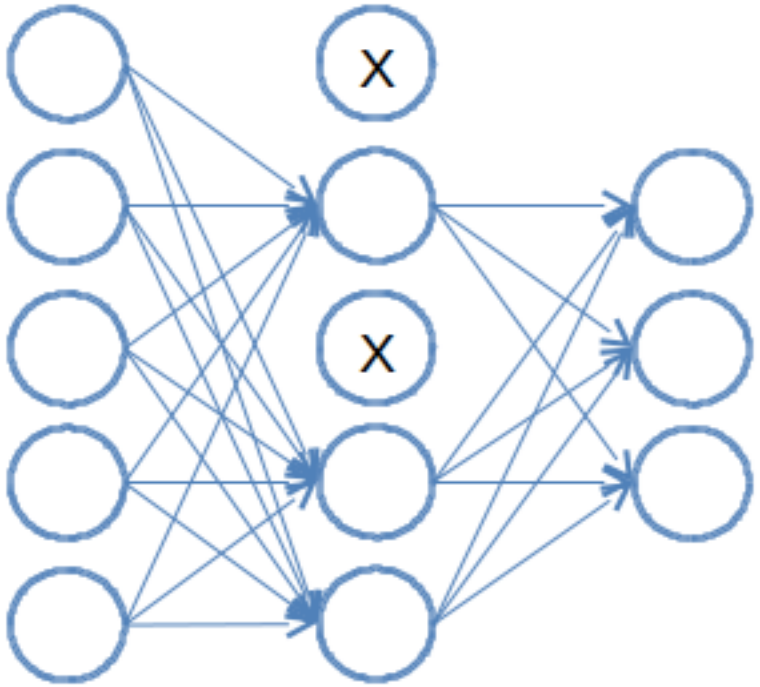
Method 2: Early Stopping



Method 3: Dropout

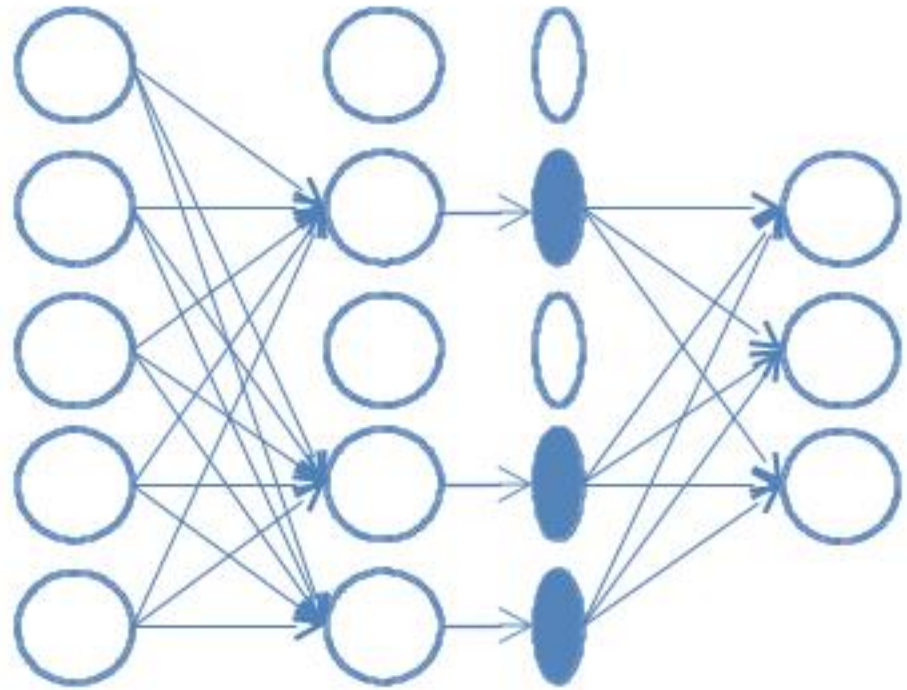


Fully Connected Network



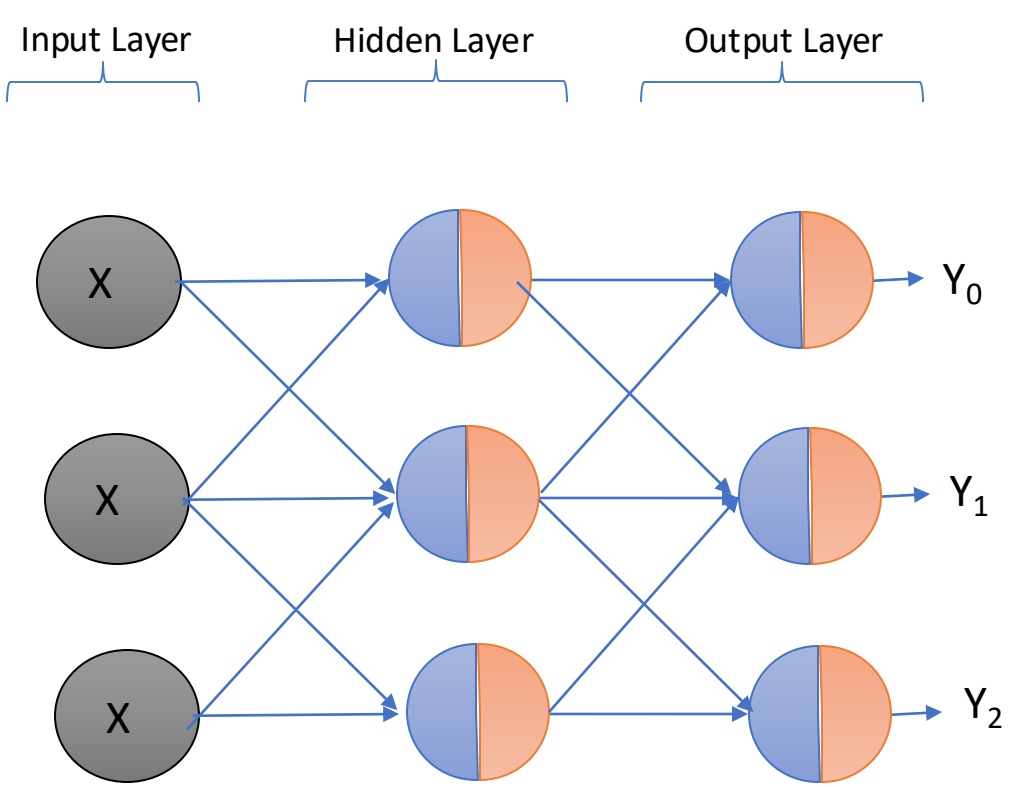
Network with Dropout in Hidden Layer

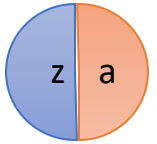
Dropout Mask



Dropout Implemented with Mask

Batch Normalization



Logit  Activation Function



For a batch of data, don't use the raw activations. Instead use 'normalized' activations over the entire match of data

The Batch Normalization Process

- 1. Calculate mean and variance for a given batch
  - We do this for each feature independently
- 2. Normalize the activations to get zero mean and unit variance

$$\hat{x}_i = \frac{x_i - \mu_{batch}}{\sqrt{\sigma^2_{batch} + \epsilon}}$$

- 3. Use learned ‘scaling factors’ to un-normalize if needed

$$y = \gamma\hat{x} + \beta$$

Training vs Inference Time

- As in dropout, we don’t have a clean transition from training to testing or inference time.

- We do not have ‘mini-batches’ during inference time, so we have no idea of what to use for mean and variance
- Instead, we use the rolling total values for these factors that are picked up during training time

Group Assignment

Split into small groups:

Use IMDB dataset and the final code block and investigate:

1) Impact of using different dropout values on training and validation loss?
2) Impact of using different activation functions instead of relu for the hidden layers?
3) Impact of introducing l2 regularization for the weights of each layer?
4) Impact of using much bigger network (double layers and number of units in each layer)